Template-Driven Reversible PLA Synthesis for Energy-Efficient Quantum and Nano-Electronic Computing

***Abstract*—Reversible PLA synthesis is a key enabler for energy-efficient quantum and nano-electronic computing, but existing flows often suffer from redundant product generation, excessive ancilla/garbage overhead, and deep critical paths. This paper presents a template-driven synthesis framework that constructs AND and EX-OR planes using reusable reversible primitives and dynamic product ordering. The method introduces Universal MUX Gate (UMG) and Universal NOR Gate (UNG) templates to realize complete minterm generation while preserving signal reuse and reducing unnecessary operations in the AND plane. Across benchmark and gate-level evaluations, the proposed flow delivers up to 50% speed improvement, about 60% area reduction, and nearly 50% lower transistor usage, while also reducing garbage outputs (up to 85%), ancilla inputs (˜62%), and gate count (22%). We further map the architecture to post-CMOS and quantum realizations, showing that the same template set scales across implementation fabrics with favorable quantum cost and routing efficiency. These results establish a compact, scalable, and hardware-conscious pathway for practical reversible PLA deployment.**

***Index Terms*—RPLA, Quantum Cost, UNG, UMG, Ancilla Inputs, Garbage Outputs, IoT, Post-CMOS, FPGA**

I. Introduction

Irreversible logic pays an energy tax: erasing one bit of information is tied to a minimum dissipation on the order of $kT \ln 2$ [1]. Reversible computation removes that fundamental loss by preserving information flow and has become a natural bridge between classical low-power design and emerging optical, post-CMOS, and quantum platforms [2]. Programmable regular structures such as programmable logic arrays (PLAs) remain attractive for constrained systems—including sensing and edge-class workloads [3]—because they offer dense, predictable layout at the cost of a structured AND/XOR decomposition. Carrying that regularity into the reversible domain yields the reversible PLA (RPLA), but existing constructions often inflate ancilla inputs, garbage outputs, and critical-path depth while duplicating product generation in the AND plane.

This work addresses that gap with a *template-driven* RPLA synthesis flow built around two new 3×3 reversible primitives—the Universal MUX Gate (UMG) and Universal NOR Gate (UNG) that realize two-literal AND-style product formation without introducing exclusive-OR in the AND plane, while propagating primary literals for reuse along a nanoscale-style routing grid. A dynamic product-ordering procedure schedules when literals are merged, swapped between rails, and recycled, so that the AND plane favors signal reuse and avoids redundant EX-OR work. The same template library then lifts to post-CMOS (memristor-oriented) and quantum descriptions, so resource metrics can be compared across fabrics with one coherent architecture.

Contributions.

- UMG/UNG templates and a grid-based AND/XOR-plane construction with formal properties for product generation and XOR composition (Sec. II).
- Algorithms for dynamic product ordering, literal swapping, and AND/XOR plane synthesis.
- Technology-oriented mappings to reversible post-CMOS PLA (Sec. III) and quantum-style circuits (Sec. IV).
- Power, critical-path, and benchmark-level comparisons against classical and prior RPLA-style baselines (Secs. V and VI), including PTL gate-level characterization of the proposed primitives.

The remainder of the paper follows that trajectory: Sec. II develops the gates, templates, and algorithms; Secs. III and IV describe post-CMOS and quantum realizations; Sec. V compares power and critical-path trends with a classical PLA; Sec. VI reports experimental metrics; Sec. VII summarizes the results and outlines future work.

II. Proposed Design of Reversible PLA

We introduce UMG/UNG primitives, Template-driven AND/XOR planes under a nano-architectonic framework, dynamic product ordering, and construction algorithms (overview: Fig. 1).

A. New Reversible Gates

1) Universal MUX Gate (UMG): The input and output vectors of Universal MUX gate can be written as (a, b, c) and (p= a, q= $b\oplus c$, r= $ab\oplus a'c$), respectively. The equivalent quantum representation of UMG is shown in Fig. 2. The proposed UMG gate performs AND operation of two literals (or a literal and a product) and also returns primary inputs to output. The unique input-output mapping of UMG gate is essential for reversible logic design allowing for the reversibility of the computation. The quantum cost of UMG is 5, making it more efficient for use in reversible PLA designs.

UMG with $(p, q, r) = (a,\ b \oplus c,\ ab \oplus \overline{a}c)$ is functionally complete: $r|_{c=0} = a \wedge b$, $q|_{b=1,c=a} = \overline{a}$, and $r|_{b=1} = a \vee c$, so $\{\wedge, \vee, \neg\} \subseteq \langle \mathrm{UMG} \rangle$.

Also $r_{\mathrm{UMG}} = ab \oplus \overline{a}c = ab + \overline{a}c$ (disjoint terms), i.e. a 2:1 MUX with select a and data b, c.

2) Universal NOR Gate (UNG): In Boolean logic, NOR gate is an universal primitives can interpret the functionalities of all basic gates (AND, OR, NOT). Similarly, the input and output vectors of proposed 3×3 Universal NOR gate which perform NOR operation can be written as (a, b, c) and (p= a, q= b, r= $(a+b)\oplus c$), respectively (shown in Fig. 3). UNG

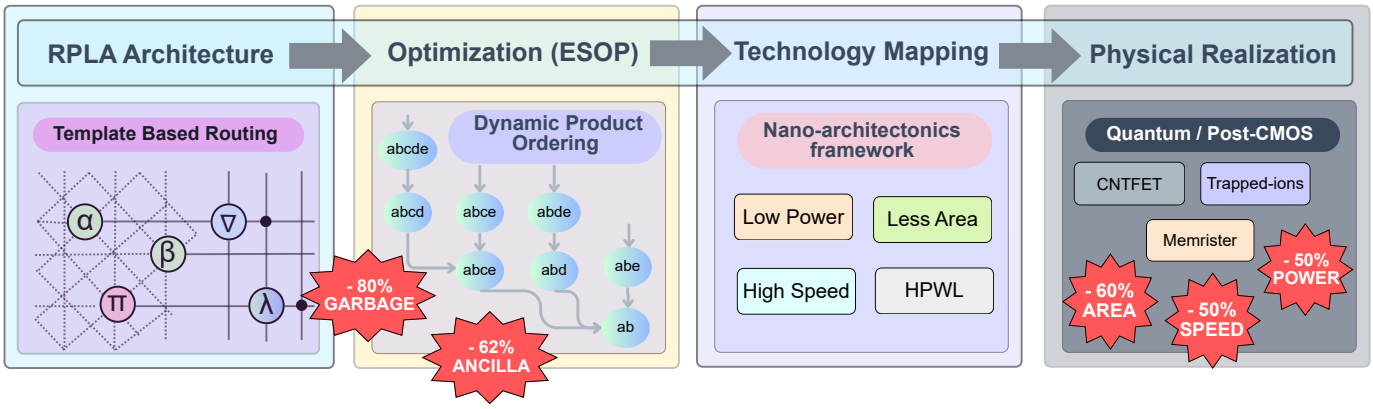
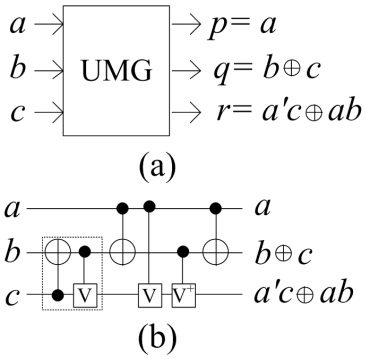


Fig. 1. Graphical overview of scalable reversible PLA architectures for quantum and post-CMOS computing.



Input			Output		
a	*b*	*c*	*p*	*q*	*r*
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	0
0	1	1	0	0	1
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1

Fig. 2. Reversible Universal MUX gate (UMG): (a) Block diagra and (b) Quantum realization; Truth table of UMG maps uniquely all the input states to output states

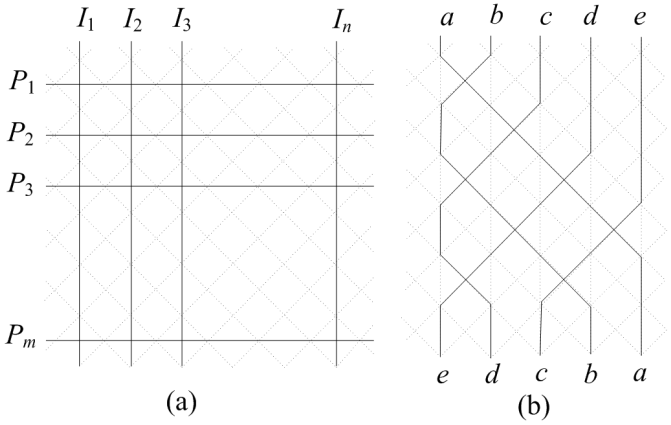
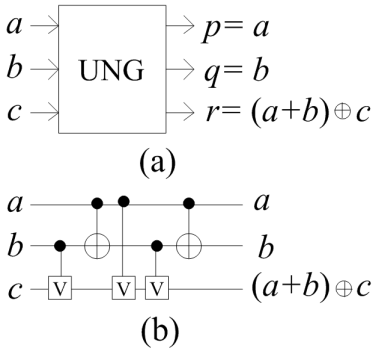


Fig. 4. Reversible PLA Grid Architecture: (a) Primary inputs and generated products be passed through vertical and horizontal lines, respectively and (b) Swapping input signals

is functionally complete: $r|_{c=0} = a \vee b$, $r|_{b=0,c=1} = \overline{a}$, and AND via $\overline{\mathrm{OR}}(\overline{a}, \overline{b})$, so $\{\wedge, \vee, \neg\} \subseteq \langle \mathrm{UNG} \rangle$.

B. Template-driven AND/XOR planes

UMG and UNG realize two-literal AND (literal or product); template choice follows I/O orientation and whether the plane produces a product or XOR sum. The AND plane instantiates those choices on a *nanoscale reversible routing grid* (Fig. 4):



Input			Output		
a	*b*	*c*	*p*	*q*	*r*
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	0	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	1	0

Fig. 3. Reversible Universal NOR gate (UNG): (a) Block diagram of UNG and (b) Quantum cost is 5; UNG maps input and output states uniquely

vertical rails carry primary literals (and aligned ancillae where needed), horizontal wires carry partial and final products, and each crosspoint binds a UMG/UNG macro so merges remain bijective and unused outputs can feed neighboring gates. Part (a) of the figure stresses orthogonal flow—inputs on vertical tracks and products on horizontal lines—while part (b) shows *literal swapping* between rails once their joint products along the grid are finished, reordering physical tracks without altering logic values. Solid and dotted segments mark default and alternate routing polarities, respectively corresponding to unprimed and primed template wiring ($\alpha, \beta, \gamma, \pi$ and $\alpha', \beta', \gamma', \pi'$); Sec. II-C formalizes the swap step (Alg. 1) and scheduling products generation on this fabric based on dynamic product ordering.

UMG uses *a* as Multiplexer (MUX) select and [*b*,*c*] as data. UMG is able to generate three minterms of two inputs (*ab*, *ab*′ and *a*′*b*, are represented through templates α, β and γ (corresponding input literal swapping orientations are α', β' and γ') as shown in Fig. 5. UMG doesn't erase the primary input value of AND Plane while generating products and those propagated unused outputs can be used as the primary inputs

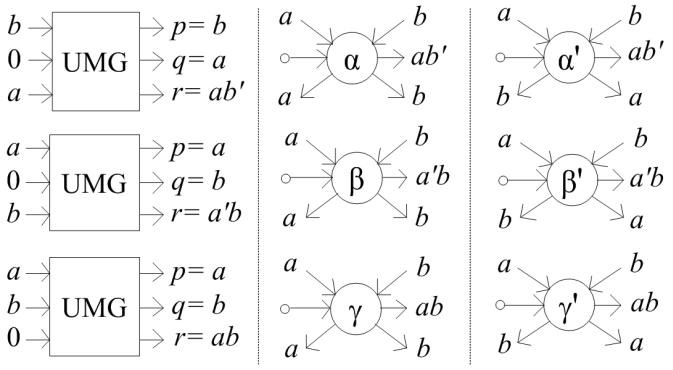


Fig. 5. Proposed templates of UMG are symbolized through α, β and γ whereas the swapping templates are α', β' and γ'

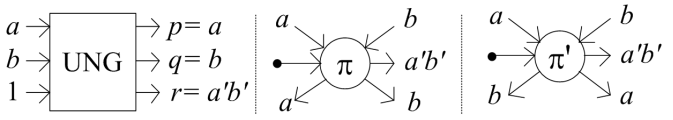


Fig. 6. Templates of UNG are used to generate (a′b′)

Algorithm 1 SwapLiterals(L_i, L_j)

1: Swap signals on L_i and L_j

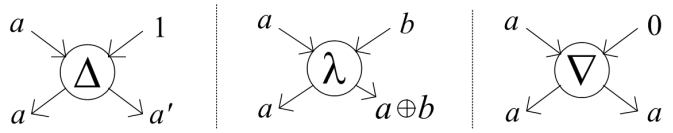


Fig. 7. Templates of CNOT are used to design EX-OR plane

by other templates (reversible gates).

UNG recovers the limitation of UMG and the operational template of UNG is symbolized using π (π') (Fig. 6).

The XOR plane uses only the templates of Feynman (CNOT) gate as $\triangle$ (NOT), λ (XOR), ∇ (COPY) [4] (Fig. 7).

Lemma 1 (Completeness of Template Set)**.** *Templates of UMG and UNG* $\{\alpha, \beta, \gamma, \pi\}$ *realize* ab, $a\bar{b}$, $\bar{a}b$, $\bar{a}\bar{b}$*; primed variants* $\{\alpha', \beta', \gamma', \pi'\}$ *are wire-isomorphic and cover the same minterms.*

C. Dynamic Products Ordering

Dynamic ordering of products reduces cost via reutilization of garbage and selecting proper templates; primary literals also follow the input order (e.g. generate *abcd* before other subsets of *abcd* minterms sharing the same tracks). Algorithm 2 sorts by $|P_i|$ and enqueues each product once when it first matches a literal in I_v.

Fig. 8 shows literal order for *a–e* where products in the same group (for example [*abcde*, *abce*, *abde*, *abe*]) are independent and can be generated in any order. In the AND plane (Fig. 4), solid (dotted) tracks mark default (alternate) paths; Alg. 1 swaps L_i, L_j after their AND operations on those tracks ends, preserving Boolean values.

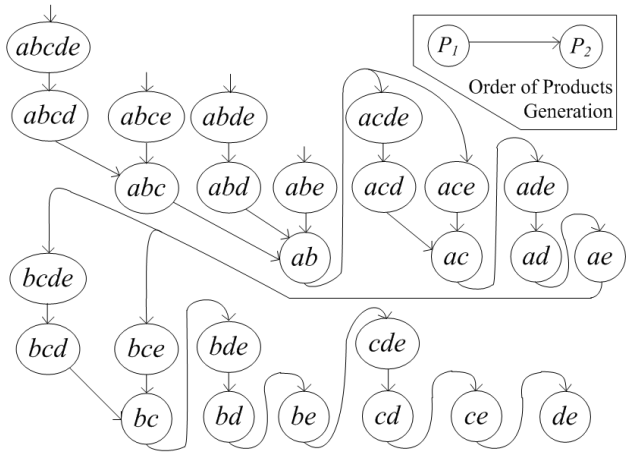


Fig. 8. In this example, the order of products consist of literals (*a*, *b*, *c*, *d*, *e*) is: {[*abcde*, *abce*, *abde*, *abe*], [*abcd*],[*abc*, *abd*], [*ab*], [*acde*, *ace*], [*acd*], [*ac*], [*ade*], [*ad*], [*ae*], [*bcde*, *bce*], [*bcd*], [*bc*], [*bde*], [*bd*], [*be*], [*cde*], [*cd*], [*ce*], [*de*]}. Products in the same group (for example [*abcde*, *abce*, *abde*, *abe*]) are independent can be generated in any order.

Algorithm 2 OrderingProducts(I_v, P_v)

Require: I_v, P_v
Ensure: Ordered P_v

```
P_Q ← ∅; sort P_v by descending |P_i|
for i = 1 to |I_v| do
    for j = 1 to |P_v| do
        if I_i ∈ P_j and P_j ∉ P_Q then
            Add P_j to P_Q
        end if
    end for
end for
P_v ← P_Q
```

D. Construction of AND and EX-OR Planes

With the help of dynamic product ordering and swapping literals, we can construct the AND plane and EX-OR plane in a more efficient way. Algorithm 4 and Algorithm 5 are the construction algorithms of AND plane and EX-OR plane, respectively.

1) Limitations of Existing RPLAs: [5] proposed the design of reversible PLA like classical [3] by using Toffoli [6] AND planes, Feynman [4] EX-OR planes, and Feynman fan-out handling in the AND plane where as [7] used one line per input literal, MUX and Feynman gates (including unnecessary copy operations in the AND plane as in [5]).

2) Designing Reversible AND Plane: AND cost dominates the PLA. Alg. 4 orders products (Alg. 2), counts *swapFlag*, calls OpAND (Alg. 3), swaps lines (Alg. 1) when no mutual products remain, and queues reusable garbage in P_{QG}. Alg. 3 maps (p, q) to UMG or UNG templates $\pi, \beta, \alpha, \gamma$; `swapFlag`=0 uses primed variants.

Algorithm 3 OpAND(p, q, swapFlag)

Map polarities to $\pi/\beta/\alpha/\gamma$; use prime templates iff `swapFlag`=0
return $p \cdot q$

Theorem 1 (AND-Plane Correctness and Resources)**.** *Algorithm 4 synthesizes all required products with UMG/UNG and preserves reversibility. Let n be the number of output AND operations and t the number of garbage ANDs in P_{QG} identical to products; then $G_{\mathrm{AND}} = n-t$, $\mathrm{QC}_{\mathrm{AND}} = 5(n-t)$, $\mathrm{AI}_{\mathrm{AND}} = n-t$.*

Proof. Let $P_v^{(0)} = \mathrm{OrderingProducts}(I_v, P_v)$ (Alg. 2). For each (g,h), if $\exists i : I_g I_h \in P_i$ then OpAND (Alg. 3, Lemma 1) extends $I_g \cdot I_h$ along I_j ($j > h$) in P_i with P_{QG}; else SwapLiterals (Alg. 1). By induction on (g,h), each line realizes $\prod_{x \in P_i} x$ bijectively. Each AND needs one UMG/UNG (cost 5) and one ancilla; t matches remove t primitives. □

Algorithm 4 ConstructANDPlane(I_v, P_v)

Require: I_v, P_v
$P_v \leftarrow$ OrderingProducts(I_v, P_v); $P_{QG} \leftarrow \emptyset$; $ndot \leftarrow 0$
for $g = 1$ to $|I_v| - 1$ **do**
  for $h = g+1$ to $|I_v|$ **do**
    $swapFlag \leftarrow |\{i : I_g I_h \in P_i\}|$
    if $swapFlag > 0$ **then**
      for $i = 1$ to $|P_v|$ **do**
        if $|P_i| > 1$ **then**
          if $P_i \in P_{QG}$ **then**
            Remove P_i from P_{QG}
          else
            if $I_g I_h \in P_i$ **then**
              $swapFlag \leftarrow swapFlag - 1$
            end if
            $pivotP \leftarrow$ OpAND($I_g, I_h, swapFlag$)
            if $|P_i| > 2$ **then**
              for $j = h+1$ to $|I_v|$ **do**
                if $I_j \in P_i$ **then**
                  $P_g \leftarrow pivotP$; $pivotP \leftarrow$ OpAND($pivotP, I_j, 0$)
                end if
              end for
              Add P_g to P_{QG}
            end if
          end if
        else
          $ndot \leftarrow ndot + 1$
        end if
      end for
    else
      SwapLiterals(L_g, L_h)
    end if
  end for
end for

3) Construction of EX-OR Plane: Alg. 5 EX-ORs products onto outputs with Feynman gates; λ ties product P_i to function F_j and keeps a copy of P_i.

Theorem 2 (XOR-Plane Correctness and Resources)**.** *Algorithm 5 XOR-combines products on each output with Feynman templates. With n XOR operations, m outputs, and $xdot$ cross-points, $G_{\mathrm{XOR}} = n + m - xdot$, $\mathrm{QC}_{\mathrm{XOR}} = n + m - xdot$, $\mathrm{AI}_{\mathrm{XOR}} = m - xdot$.*

Proof. Outputs $F_j = \bigoplus \tilde{P}_i$ via $(x,y) \mapsto (x, x \oplus y)$; FreqOf tracks copies; $\triangle, \nabla, \lambda$ are bijective. Gate count is $n + m$ merges minus $xdot$ shared cross-points. □

4) Functional and Resource Analysis: Algs. 4 and 5 realize F in Fig. 9 (visualizes interactions among total gates, garbage outputs, and ancilla inputs). Table I aggregates GA, GB, AI, and QC for the same F against the prior Reversible PLA [7]. Each output has an Exclusive-SOP; Lemma 1 and Theorem 1 build products and Theorem 2 XORs them, so the architecture is universal for multi-output Boolean functions.

$$F = \begin{cases} f_1 = ab' \oplus ab'c & f_2 = ac \oplus a'b'c & f_4 = ac \\ f_3 = ab' \oplus ab'c \oplus bc' & f_5 = ab' \oplus ac \oplus bc' & \end{cases} \tag{1}$$

The Function F in Eq. (1), n=7, t=1, so gates $n-t$=6, quantum cost 30, ancilla 6 (Fig. 9). With p multi-literal products, q garbage lines duplicating products, L=totalLiterals, and $ndot$ cross-points, garbage count obeys $G_{\mathrm{GB}} = p + L - ndot - q$.

Algorithm 5 ConstructXORPlane(P_v, F_v)

Require: P_v, F_v
$F_Q \leftarrow \emptyset$; $xdot \leftarrow 0$
for $i = 1$ to $|P_v|$ **do**
  for $j = 1$ to $|F_v|$ **do**
    if $P_i \in F_j$ **then**
      if $F_j \notin F_Q$ **then**
        if FreqOf(P_i) $= 1$ **then**
          $xdot \leftarrow xdot + 1$
        else
          preserve P_i; FreqOf(P_i) $\leftarrow$ FreqOf(P_i) $- 1$
        end if
        Add F_j to F_Q
      else
        XOR-map P_i to F_j
      end if
    else if $P_i' \in F_j$ **then**
      preserve complemented P_i
    end if
  end for
end for

TABLE I
PROPOSED VS. EXISTING [7] RPLA FOR MULTI-OUTPUT F (EQ. 1).

RPLA design	GA	GB	AI	QC
Proposed	13	5	7	37
Existing [7]	18	10	12	39

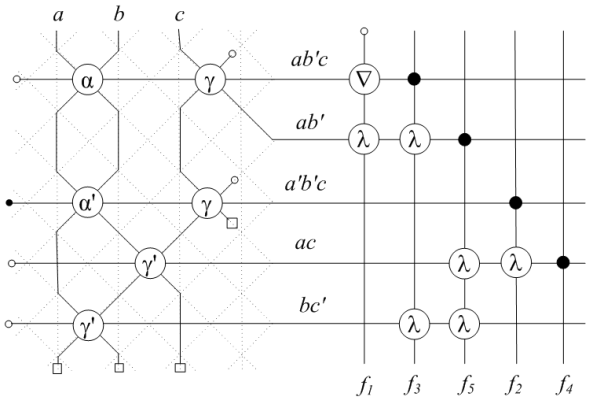


Fig. 9. Optimized version of Reversible PLA of multi-output function F in Equation(I)

III. PROPOSED REVERSIBLE POST-CMOS PLA

We propose a post-CMOS realization of the RPLA design, mapping UMG to CNTFET-based transmission gates and UNG to memristor crosspoints, preserving bijective I/O for uncomputation and garbage recycling in the PLA grid.

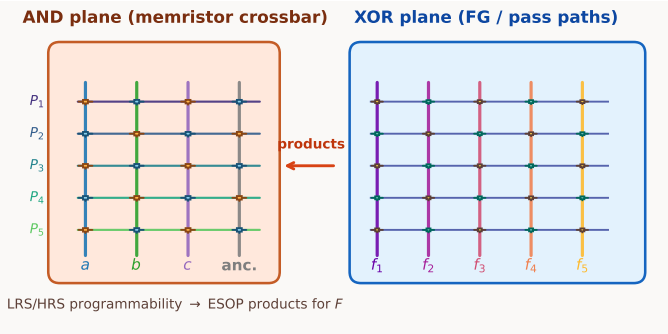
A. Device mapping for UMG and UNG

The AND and EX-OR planes of Sec. II are compositions of UMG, UNG, and Feynman primitives; each composition inherits a post-CMOS layout by substituting CNTFET-based cells for UMG paths and memristor crosspoints for UNG-style combines, following the mappings in the bullets below.

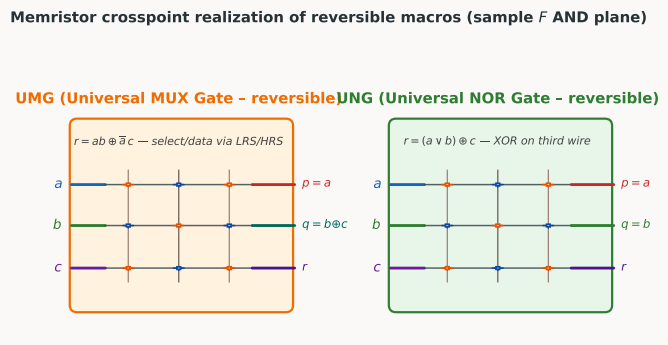
- **CNTFET paths** implement low-resistance transmission-gate style routing and restoration suited to the fan-out-preserving outputs of UMG (p, q copies of inputs).
- **Memristor crosspoints** programmable high/low resistance for conditional paths, matching UNG-style combine XOR behavior where the control $(a \vee b)$ steers inversion on c.

B. Reversible embedding and composition

Any incompletely specified Boolean $f : \{0,1\}^n \to \{0,1\}^m$ embeds in a reversible map on $k \geq n+m$ lines by adding ancilla and garbage [2]. Concretely, one may use a Bennett-style embedding so that the target outputs appear on y while x and ancilla a remain clean for reuse. This closes the loop from algorithm (Algs. 4–5) to a technology-mapped reversible netlist suitable for SPICE/Liberty-driven signoff, as in the PTL study (Table IV).



(a) Regular memristor crossbar RPLA for F: AND plane (programmable LRS/HRS) and XOR/FG plane to f_j.



(b) Memristor crosspoint realization of UMG (MUX paths) and UNG (OR⊕c) macros used in the AND plane.

Fig. 10. Two post-CMOS memristor realizations of sample F (Eq. 1): (a) uniform crossbar without explicit macro tiling; (b) UMG/UNG memristor cells composing the same logic.

C. Memristor crossbar RPLA for sample F

Memristive crosspoints provide programmable low vs. high resistance (LRS/HRS) suitable for conditional routing in [8]. For the sample multi-output F (Eq. 1, Fig. 9), we distinguish two memristor-centric realizations; Fig. 10 summarizes both.

(a) Regular memristor RPLA. Figure 10(a) shows a *generic* crossbar-style AND plane: vertical rails carry inputs and ancilla, horizontal lines are abstract product wires P_i, and each crossing is a memristor programmed to LRS or HRS to realize the ESOP support for F without expanding the grid into named reversible macros.

(b) Memristor implementation of UMG and UNG. Fig. 10(b) refines the AND-plane primitives: each UMG and UNG is a compact 3×3 *reversible macro* wired as in Sec. II-A–II-B, with internal memristor crosspoints enacting the MUX select for UMG ($(p, q, r) = (a,\ b \oplus c,\ ab \oplus \bar{a}c)$) and the OR-then-XOR semantics for UNG ($(p, q, r) = (a,\ b,\ (a \vee b) \oplus c)$). Technology mapping places these cells on the memristor fabric; the EX-OR plane is unchanged from (a).

Toolchain (Qiskit-like workflow). Fig. 10 shows the `MEMXP`-based netlist (LRS/HRS crosspoints with ESOP/Bennett scheduling) is simulated via `ngspice -b` and validated over all 8 input patterns against F before netlist generation.

IV. Proposed Reversible Quantum PLA

We propose a quantum realization of the RPLA design, mapping UMG and UNG to unitary operators on qubits and Feynman templates to CNOT gates, preserving reversibility and enabling hierarchical quantum compilation with low overhead.

A. Unitary view of the Reversible PLA

Every wire in the reversible PLA carries a qubit in the computational basis during classical reversible simulation; each UMG, UNG, or Feynman template is a permutation matrix and hence a unitary operator on those qubits [9]. The full PLA is therefore a structured product of such operators, acting on a tensor-product Hilbert space $\mathcal{H}^{\otimes k}$ with k the number of lines (inputs, ancillae, and garbage routed explicitly). Product ordering from Sec. II-C induces a *layered* quantum circuit: AND-plane chains products, then XOR-plane Feynman gates accumulate parities on output registers.

B. Primitive gates and quantum cost

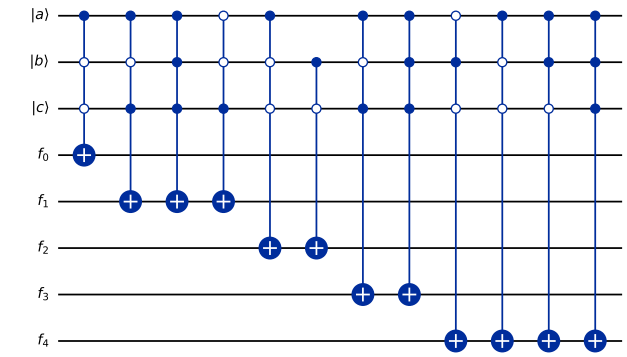
The abstract quantum costs used in Sec. II-C (UMG/UNG cost 5, Feynman cost 1) align with standard reversible-gate libraries [4], [6]. In a fault-tolerant or NISQ compilation stack, each macro-gate is further decomposed into a universal set (e.g., CNOT plus single-qubit rotations, or Toffoli+Hadamard families) while preserving the same I/O semantics. Hierarchical quantum CAD flows [10] support exactly this: high-level reversible blocks are expanded to native gates subject to depth, fidelity, and ancilla budgets—analogous to technology mapping in the post-CMOS flow above.

C. Qiskit realization of sample F

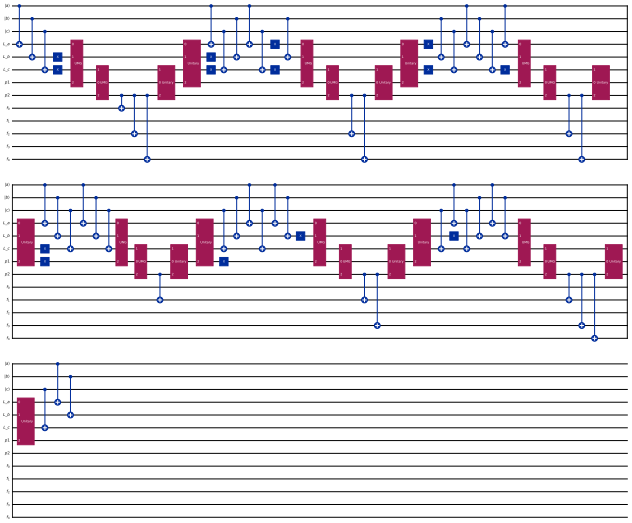
The same multi-output F from Eq. (1), optimized in the reversible PLA layout of Fig. 9, admits Bennett-style quantum embeddings on Qiskit. **(a)** Fig. 11(a) compacts the ESOP into multi-controlled X gates (mixed `ctrl_state` values) on $|a\rangle$, $|b\rangle$, $|c\rangle$ with five output ancillae $\mathtt{f}_i \leftarrow f_i$. **(b)** Fig. 11(b) matches the Sec. II-C template story: the AND plane uses the same 3×3 permutation macros as the quantum realizations of UMG and UNG in Figs. 2–3 (UMG for two-literal AND; UNG for the $a'b'$ factor in minterm $a'b'c$ via $(a \vee b) \oplus 1$ on an ancilla initialized to $|1\rangle$, i.e. classical $(a \vee b)' = a'b'$); literals are copied with CNOT and the EX-OR plane uses the Feynman template (FG: λ) as controlled-X from each product ancilla onto $\mathtt{f}_j$, with Bennett uncompute of ancillae between minterms.

D. Realization outlook

The proposed realization path is: (i) synthesize the RPLA as a reversible netlist (Sec. II); (ii) map primitives to quantum circuits preserving ancilla/garbage discipline [2]; (iii) optimize depth across AND and EX-OR planes using the shared-product structure from Sec. II-D. This positions the same template set for either post-CMOS classical emulation or quantum execution, with resource metrics (GA, GB, AI, QC) carrying over as first-order proxies for width, ancilla count, and two-qubit gate volume.



(a) ESOP / multi-controlled-X form (compact).



(b) AND plane: UMG and UNG macros; EX-OR plane: Feynman (CNOT) into `f` lines; ancillae $\mathtt{L}_*$, $\mathtt{p}_1$, $\mathtt{p}_2$.

Fig. 11. Qiskit realizations of F (Eq. 1), aligned with Fig. 9: (a) ESOP embedding; (b) UMG/UNG AND templates plus Feynman XOR (same Boolean map).

V. Power and Critical-Path Comparison with Classical PLA

We compare four classes at the same activity/technology corner (Nangate 45 nm, V_{DD}=1.8 V): non-reversible PLA, reversible PTL PLA, reversible post-CMOS outlook, and reversible quantum proxy. For non-reversible PLA, $G_{\mathrm{NR}} = \max\left(8, \lceil 0.38\,\mathrm{GA}^{\S} + 0.22\,\mathrm{GA}^{\dagger} \rceil\right)$ estimates two-level size; power is plane switching plus input recovery, $P_{\mathrm{NR}} = G_{\mathrm{NR}}(0.82{\times}10^{-4}) + \mathrm{AI}^{\dagger}(0.58{\times}10^{-4})$ W, capturing re-buffer/re-latch overhead when literals are not preserved [7], [11]. Reversible PTL power is $\mathrm{GA}^{\dagger} \cdot 1.45{\times}10^{-4}$ W (Table IV); post-CMOS uses illustrative scaling $(\eta_{\mathrm{post}}, \gamma_{\mathrm{post}}) = (0.86, 0.92)$; quantum uses $P_{\mathrm{Q}} = \kappa\,\mathrm{QC}^{\dagger}$ with $\kappa = 0.32\,P_{\mathrm{macro}}$ and $\tau_{\mathrm{Q}} \propto (\mathrm{QC}^{\dagger})^{0.36}$. Critical-path trends use $\tau \propto G^{0.44}$ as a first-order model.

Fig. 12 shows these four classes as line traces (top: log-power, bottom: critical path), and Table II lists the numeric values. The reversible cases reuse the same RPLA template

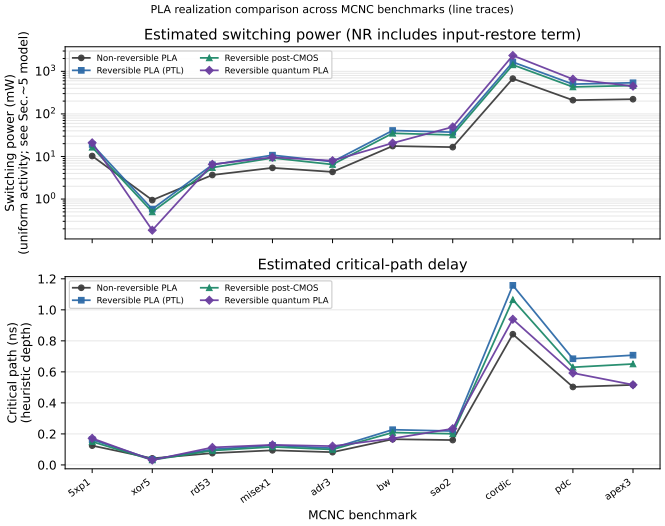


Fig. 12. Four realization classes per benchmark (line traces): **(top)** switching power (log scale; non-reversible total includes input-recovery term, Sec. V); **(bottom)** critical-path delay. *Non-reversible* AND–OR PLA; *Reversible PLA* (PTL UMG/UNG, GA†); *Reversible post-CMOS* (memristor/CNTFET illustrative scale); *Reversible quantum PLA* (QC† proxy). Same MCNC subset as Table II; models in Sec. V [3], [11].

from Secs. III–IV (post-CMOS and quantum interpretations of the same UMG/UNG+Feynman structure).

VI. Performance Analysis

Table III reports GA, GB, AI, and QC for proposed (†) versus existing [7] (§). Across benchmarks, the proposed flow consistently reduces GA/GB/AI through product ordering and reuse (Theorems 1, 2); QC is benchmark-dependent because fewer lines/gates do not always minimize two-qubit decompositions. Fig. 13 plots GA together with GB and AI to highlight the reversible-resource tradeoffs suggested by the table. Fig. 14 summarizes QC and GB across the same four realization classes as Fig. 12 (NR shown as zero where reversible metrics are not defined).

The obtained trends are consistent with the AND/XOR resource bounds in Theorems 1 and 2. At gate level, PTL UMG/UNG (Table IV) keep bijective mappings, quantum cost 5, and improve area/power/delay against listed baselines; Fig. 15 visualizes these gains.

TABLE II
Estimated switching power and critical-path delay: non-reversible PLA, reversible PTL PLA (GA†), reversible post-CMOS (illustrative η_{post},γ_{post}), and reversible quantum PLA (QC† proxy; Sec. V).

Bench.	P_{NR} **(mW)**	P_{PLA} **(mW)**	P_{post} **(mW)**	P_{Q} **(mW)**	$\mathbf{CP}_{\text{NR}}$ **(ns)**	$\mathbf{CP}_{\text{PLA}}$ **(ns)**	$\mathbf{CP}_{\text{post}}$ **(ns)**	$\mathbf{CP}_{\text{Q}}$ **(ns)**
5xp1	10.27	19.14	16.46	20.97	0.12	0.16	0.15	0.17
xor5	0.95	0.58	0.50	0.19	0.04	0.03	0.03	0.03
rd53	3.68	6.38	5.49	6.54	0.08	0.10	0.09	0.11
misex1	5.41	10.73	9.23	9.56	0.09	0.13	0.12	0.13
adr3	4.34	7.54	6.48	7.98	0.08	0.11	0.10	0.12
bw	17.59	40.75	35.04	20.65	0.17	0.23	0.21	0.17
sao2	16.58	37.27	32.05	49.42	0.16	0.22	0.20	0.23
cordic	674.43	1651.40	1420.21	2355.50	0.84	1.16	1.07	0.94
pdc	210.28	500.40	430.34	654.94	0.50	0.68	0.63	0.59
apex3	221.56	539.25	463.76	447.99	0.52	0.71	0.65	0.52

P_{NR}: G_{NR} plane switching at $\sim 0.82\times10^{-4}$ W/node plus input-recovery $\text{AI}^{\dagger}\cdot 0.58\times10^{-4}$ W (Sec. V). PLA: CMOS PTL UMG/UNG at GA†; post: $\times 0.86$ power, $\times 0.92$ CP vs. PLA; Q: QC† proxy (not signoff mW).

TABLE III
Experimental results for various benchmark functions.

B.M.	**GA§ (GA†)**	**GB§ (GB†)**	**AI§ (AI†)**	**QC§ (QC†)**
5xp1	166 (132)	112 (44)	115 (47)	418 (452)
9sym	427 (377)	385 (111)	377 (103)	1405 (1681)
adr3	67 (52)	48 (26)	46 (24)	157 (172)
apex3	3998 (3719)	1799 (520)	1795 (516)	8654 (9655)
b12	159 (127)	132 (56)	167 (50)	453 (495)
bw	305 (281)	64 (30)	87 (53)	446 (445)
cordic	12162 (11389)	10640 (1573)	10661 (1552)	41694 (50765)
duke2	941 (874)	667 (157)	674 (164)	2735 (3262)
e64	2170 (2074)	2148 (121)	2148 (121)	8410 (10282)
inc4	16 (11)	10 (4)	11 (5)	34 (35)
inc5	23 (16)	16 (6)	17 (7)	53 (56)
misex1	88 (74)	51 (25)	50 (24)	193 (206)
misex2	199 (165)	176 (79)	149 (52)	122 (673)
pdc	3801 (3451)	3006 (487)	3030 (511)	12096 (14115)
rd53	56 (44)	42 (23)	40 (21)	134 (141)
rd73	187 (150)	141 (68)	137 (64)	487 (550)
rd84	328 (267)	265 (114)	261 (110)	928 (1067)
sasao	14 (9)	13 (8)	10 (5)	29 (29)
sao2	284 (257)	236 (60)	230 (54)	890 (1065)
t481	49 (40)	53 (36)	38 (21)	134 (152)
table3	2602 (2439)	1814 (342)	1814 (342)	7537 (8995)
table5	2539 (2384)	1819 (319)	1682 (182)	7516 (8896)
vag2	2018 (1835)	1836 (383)	1770 (317)	6926 (8379)
xor5	8 (4)	8 (9)	4 (5)	8 (4)
z5xp1	167 (139)	114 (50)	117 (53)	425 (483)
z9sym	427 (377)	385 (111)	377 (103)	1405 (1681)

§Existing and †Proposed RPLA designs.

VII. Conclusion

We presented a template-driven reversible PLA (RPLA) flow centered on two 3×3 primitives—UMG and UNG—that support two-literal product formation in the AND plane while preserving primary literals for local reuse on a structured routing grid. Dynamic product ordering, together with explicit literal swapping and XOR-plane composition, reduces redundant work and tightens reversible resources relative to a representative prior RPLA baseline [7]. The same macro-level templates map cleanly to pass-transistor-style implementations, to post-CMOS memristor/CNTFET-style sketches, and to layered quantum circuits, which lets designers compare gate count, ancilla/garbage load, quantum cost, and first-order power or delay proxies within one architectural story.

Benchmark and gate-level experiments in Sec. VI corroborate these trends: the proposed flow reports sizable reductions in garbage outputs, ancilla inputs, and gate count on standard MCNC-style functions, alongside competitive or improved area, delay, and power characteristics for the characterized UMG/UNG cells (Table IV, Fig. 15). Cross-fabric

TABLE IV
METRICS BASED ON PASS TRANSISTOR LOGIC (PTL) IMPLEMENTATION OF REVERSIBLE GATES

Reversible Gates	Area (μm^2)	Wire-length μm)	Avg. Power (10^{-4}W)	HPWL(μm)	Rise time t_{pLH}(ps)	Fall time t_{pLH}(ps)
Feynman Gate [4]	63	42	0.77452	595467	2.3442	2.0262
Toffoli Gate [6]	64	179	1.2552	896062	2.1184	1.8279
Fredkin Gate [12]	64	179	1.2030	896062	2.4427	1.4886
Press Gate [9]	65	74	2.0522	907700	2.1184	1.8279
MUX Gate [13]	66	313	2.1578	898293	1.6	1.6
Proposed Universal MUX Gate	65	210	1.4402	896230	1.6	1.6
Proposed Universal NOR Gate	64	179	1.4619	896062	1.6	1.6

Toolchain: Yosys, OpenROAD, OpenSTA and ngspice; Technology: Nangate45, VDD=1.8V

comparisons in Sec. V further illustrate how the same RPLA template propagates to classical, post-CMOS outlook, and quantum proxy models, while acknowledging that quantum cost is benchmark-sensitive once universal decompositions and mapping constraints are applied.

Future work. Tighter coupling to industrial synthesis and placement (e.g., Verilog-to-reversible-PLA flows with incremental verification), exploration of larger and arithmetic-heavy benchmarks under fixed ancilla budgets, and signoff-oriented models for memristor variability and quantum error/fault-



Fig. 13. Gate count, garbage outputs, and ancilla inputs for reversible PLA realizations (illustrative tradeoffs).

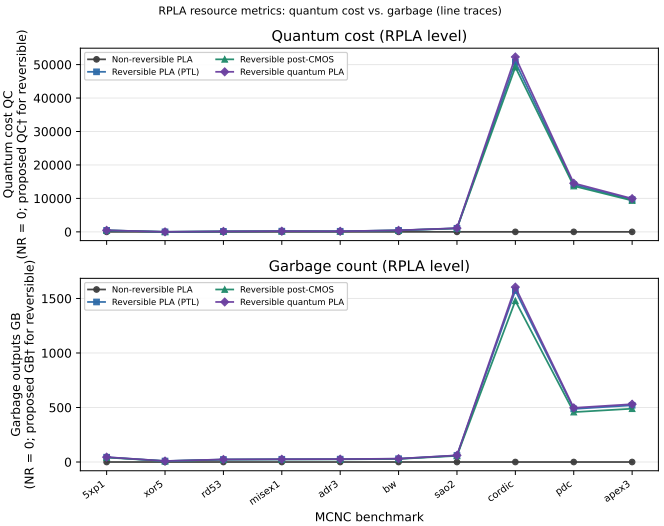


Fig. 14. RPLA metrics across four realization classes (same benchmarks as Fig. 12): **(top)** quantum cost QC† (non-reversible PLA has no reversible QC, shown as 0); **(bottom)** garbage outputs GB† (NR=0). Both panels use line traces with markers (linear axes). Reversible post-CMOS and quantum rows use illustrative scalings vs. PTL ($\times 0.97/\times 1.03$ on QC; $\times 0.94/\times 1.02$ on GB) for cross-benchmark comparison only (Sec. III–IV).

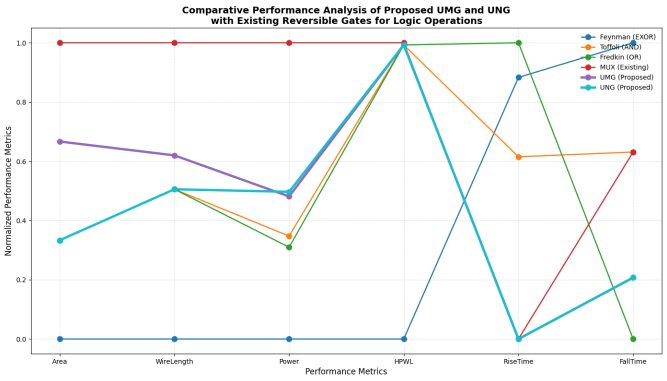


Fig. 15. The outperformance of proposed UMG and UNG gates enhances the overall performance of Reversible PLA

tolerant overhead remain open. Extending the template library for multi-fan-in product merge without sacrificing bijectivity, and validating post-CMOS layouts through full layout–parasitic extraction, would strengthen the path from algorithmic RPLA construction to deployable reversible fabrics.

ACKNOWLEDGMENT

Many thanks to the open-source EDA community for providing tools and benchmarks that enabled this research. We also acknowledge the support of our institution and colleagues who contributed valuable insights during this work.

REFERENCES

[1] R. Landauer, "Irreversibility and heat generation in the computing process," *IBM J. of R&D*, vol. 5, no. 3, pp. 183–191, 1961.

[2] C. Bennett, "Logical reversibility of computation," *IBM J. of Research and Development*, vol. 17, no. 6, pp. 525–532, 1973.

[3] H. Fleisher and L. Maissel, "An introduction to array logic," *IBM J. of Res. and Development*, vol. 19, no. 2, pp. 98–109, 1975.

[4] R. Feynman, "Quantum mechanical computers," *Foundations of Physics*, vol. 16, no. 6, pp. 507–531, 1986.

[5] A. Chowdhury, R. Nazmul, and H. Babu, "A new approach to synthesize multiple-output functions using reversible programmable logic arrays," in *Proceedings of the 19th International Conference on VLSI Design (VLSID'06)*, India, 2006, pp. 311–316.

[6] T. Toffoli, "Reversible computing," in *Lecture Notes in Comp. Sci. on Auto., Lang. and Prog.*, vol. 85, 1980, pp. 632–644.

[7] S. Mitra, L. Jamal, M. Kaneko, and H. Babu, "An efficient approach for designing and minimizing reversible programmable logic arrays," in *Proceedings of the Great Lakes Symposium on VLSI*. New York, NY, USA: ACM, 2012, pp. 215–220.

[8] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, no. 7191, pp. 80–83, 2008.

[9] A. Peres, "Reversible logic and quantum computers," *Physical Review A*, vol. 32, no. 6, pp. 3266–3276, Dec 1985. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.32.3266

[10] M. Perkowski and et al., "A hierarchical approach to computer aided design of quantum circuits," in *Proceedings of the 6th International Symposium on R&M of Future Comp. Tech.*, Germany, 2003, pp. 201–209.

[11] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Prentice Hall, 2003.

[12] E. Fredkin and T. Toffoli, "Conservative logic," *International Journal Theoretical Physics*, vol. 21, pp. 219–253, 1982.

[13] A. Sayem and S. Mitra, "Fault-tolerant reversible circuits," in *Proceedings of IEEE International Conference on Computer Science and Automation Engineering (CSAE'11)*, Shanghai, China, 2011, pp. 251–255.